Full-Stack Developer Assignment

📋 Project Overview

Build a **Task Management Application** with a RESTful API backend and a responsive frontend interface.

Tech Stack:

- Frontend: React or Next.js (your choice)
- Backend: Node.js/Express, Python/FastAPI, or your preferred framework
- Database: Your choice (SQLite, PostgreSQL, MongoDB, or in-memory)

🎯 Core Requirements (Must Complete)

Backend

1. **API Endpoints**
   - `POST /api/tasks` - Create a new task
   - `GET /api/tasks` - List all tasks with optional status filter
   - `PATCH /api/tasks/:id` - Update task status
   - `DELETE /api/tasks/:id` - Delete a task
2. **Task Model**

```json
{
  "id": "string",
  "title": "string",
  "description": "string",
  "status": "pending | in-progress | completed",
  "createdAt": "timestamp"
}
```

3. **Validation**
   - Title is required (max 100 characters)
   - Status must be one of the allowed values
   - Return appropriate error messages and HTTP status codes
4. **Basic Error Handling**

- Handle 404 for non-existent tasks
- Handle validation errors with clear messages

Frontend

1. **Task List View**
   - Display all tasks in a clean layout
   - Show task title, description, and status
   - Visual distinction between different statuses
2. **Create Task Form**
   - Input fields for title and description
   - Form validation before submission
   - Clear success/error feedback
3. **Task Actions**
   - Button to change task status (pending → in-progress → completed)
   - Delete button with confirmation
   - Disable actions during API calls
4. **Basic Styling**
   - Responsive design (mobile-friendly)
   - Clean, modern UI (Tailwind CSS, CSS modules, or styled-components)
   - Loading states for async operations

🚀 Stretch Goals (Optional Enhancements)

Choose any that interest you if you have time:

- **Filtering**: Add tabs/buttons to filter tasks by status
- **Editing**: Allow inline editing of task title/description
- **Search**: Add search functionality to filter tasks by title
- **Sorting**: Sort tasks by creation date or status
- **Priority Levels**: Add priority field (low, medium, high) with color coding
- **Due Dates**: Add due date field with visual indicators for overdue tasks

- **Pagination**: Implement pagination if you have many tasks
- **Dark Mode**: Add theme toggle
- **Unit Tests**: Add basic tests for API endpoints or components
- **Docker**: Containerize the application

📦 Deliverables

1. **Source Code**
   - GitHub repository (public or private with access)
   - Separate folders for `backend` and `frontend`
2. **README.md** must include:
   - Setup instructions (dependencies, environment variables)
   - Commands to run backend and frontend
   - API documentation (endpoints, request/response examples)
   - Any assumptions or decisions you made
3. **Running Application**
   - Backend should run on `http://localhost:5000` (or specify port)
   - Frontend should run on `http://localhost:3000` (or specify port)
   - Both should be able to run simultaneously

✅ Evaluation Criteria

We'll assess:

- **Functionality** (40%): Core features work as expected
- **Code Quality** (30%): Clean, readable, well-organized code
- **UI/UX** (15%): Intuitive interface and user feedback
- **Documentation** (15%): Clear setup instructions and API docs

We value:

- Working software over perfect software
- Practical decisions over over-engineering

- Clear communication in code and documentation

💡 Tips

- **Start simple**: Get core features working before attempting stretch goals
- **Use what you know**: Choose familiar tools to maximize efficiency
- **Commit regularly**: Shows your thought process and progress
- **Ask questions**: If requirements are unclear, document your assumptions in the README
- **Test as you go**: Ensure each feature works before moving to the next

---

📧 Submission

Please submit your completed assignment by sharing the GitHub repository link.

Questions? Feel free to reach out if you need any clarifications.

Good luck! We're excited to see your approach to this challenge. 🚀